UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERÍA

Escuela Profesional de Ingeniería de Sistemas

“Plataforma web para la comercialización y revalorización de productos perecibles orientada al consumo sostenible en la provincia de Tacna”

Curso: Construcción de Software I

Docente: Mag. Ricardo Eduardo Valcárcel Alvarado

Integrantes:

Renzo Fernando Loyola Vilca ***(2021072615)***

Brant Antony Chata Choque ***(2020067577)***

Tacna – Perú

2026

1. Introducción y metodología

El presente informe analiza la trazabilidad del Sprint 6 de ÓrbiKa, denominado “Confianza y perfil”, planificado del 2 al 7 de noviembre de 2026, con una capacidad estimada de 6 puntos de historia. Su objetivo es permitir una reputación basada en pedidos completados y mantener los datos del titular de la cuenta actualizados. Se evalúan RF-09 (calificar transacción) y RF-14 (editar perfil de usuario).

La revisión relaciona los requisitos comprometidos con la especificación de requisitos (SRS), el código real del repositorio y las pruebas automatizadas que pudieron ejecutarse. A diferencia de los informes de los Sprints 3, 4 y 5 — donde la mayor parte de lo planificado ya estaba implementado — esta revisión encontró una diferencia importante entre lo planificado y lo construido para RF-14, que se detalla en la sección 4.

2. Alcance y compromiso del Sprint 6

Campo	Definición según la planificación
Nombre	Sprint 6 — Confianza y perfil
Periodo	2–7 de noviembre de 2026
Capacidad	6 puntos de historia
Objetivo	Permitir reputación basada en pedidos completados y mantener los datos del titular actualizados.
Requisitos comprometidos	RF-09 (calificar transacción), RF-14 (editar perfil de usuario)
Dependencia	Depende de pedidos e identidad validados en los Sprints 1 y 4.
Entregable	Perfil editable y registro de calificación ligado a una transacción completada.
Criterio de cumplimiento	Solo el titular modifica su perfil; solo el comprador del pedido completado califica una vez y con puntaje de 1 a 5.
Evidencia planificada	Capturas de perfil actualizado y de registro de calificación; cubrir los rechazos en el reporte de prueba.
Tareas y responsables	Renzo: habilitar la edición de perfil con campos según el rol y control de titularidad; validar la calificación entera de 1 a 5 ligada a un pedido completado. Brant: probar perfil ajeno, pedido pendiente, calificación repetida y puntajes fuera de rango.

Resumen de estado basado en la evidencia disponible

Requisito	Evidencia en el código	Estado de trazabilidad
RF-09 — Calificar transacción	calificarPedido() valida rol, pertenencia y estado completado, e inserta en la tabla calificaciones, protegida por una restricción UNIQUE sobre pedido_id.	Parcialmente implementado: el registro de la calificación existe, pero el promedio del vendedor que el SRS exige mostrar no se calcula en

Requisito	Evidencia en el código	Estado de trazabilidad
		ningún lugar del código. Cobertura de prueba real mínima.
RF-14 — Editar perfil de usuario	No se encontró ninguna ruta de backend, página de frontend ni función de servicio que permita editar el perfil. Solo existe una política de base de datos (RLS) que lo permitiría si el cliente la invocara.	No implementado. El perfil solo se lee al iniciar sesión; no existe ninguna operación de escritura desde el cliente.

3. Requerimiento — RF-09: Calificar transacción

Elemento de trazabilidad	Análisis
SRS — definición	CU-06 “Calificar transacción” (RF-09): el pedido debe pertenecer al comprador, estar en estado Completado y no tener calificación previa. Se registra una única calificación y el promedio del vendedor se calcula a partir de sus transacciones calificadas, no mediante una encuesta adicional.
Caso de uso	CU-06, disparado desde “Mis compras” sobre un pedido completado. El flujo principal tiene 8 pasos; las alternativas cubren calificación repetida (E-1/E-2), puntaje inválido (E-3/E-4) y pedido ajeno o no completado (E-5/E-6).
Evidencia backend — ruta y controlador	backend/src/routes/pedido.routes.js expone POST /api/pedidos/:id/calificacion con requireRole(“comprador”), comentado como CU-06. pedido.controller.js → calificar() delega al servicio.
Evidencia backend — servicio	pedido.service.js → calificarPedido(pedidoId, actor, puntaje, comentario, db): exige rol comprador, valida el puntaje con validarCalificacion() (entero de 1 a 5), busca el pedido y verifica que pertenezca al comprador y esté en estado completado, e inserta la fila en calificaciones; si la inserción falla, lo reporta como “este pedido ya fue calificado”.
Evidencia de base de datos	supabase/migrations/001_init_schema.sql define la tabla calificaciones con pedido_id bigint not null UNIQUE — esta restricción impide, a nivel de base de datos, más de una calificación por pedido incluso si la verificación de la aplicación fallara.
Lo que el código NO hace (brecha frente al SRS)	El paso 8 del flujo principal de CU-06 exige “recalcular el promedio del vendedor”. No se encontró ninguna vista, consulta ni campo en todo el repositorio que calcule o exponga ese promedio: producto.service.js solo trae nombre_negocio y ubicación del perfil del vendedor (sin promedio), y no existe ninguna vista SQL de calificaciones agregadas. Esto confirma, para RF-09, el mismo hallazgo ya señalado en el informe de Sprint 3 sobre el promedio del vendedor pendiente.
Pruebas reales ejecutadas	Backend (validación, pedido.validation.test.js): “limita puntaje y comentario” — PASS (acepta puntaje 1-5, rechaza puntaje 6). No se encontró ninguna otra prueba real: calificarPedido() no está cubierta por ningún mock ni por una prueba contra la base de datos real (PGlite); tampoco existe una prueba que verifique el rechazo por pedido ajeno, por pedido no completado, o la restricción de calificación única.
Criterio de aceptación	Solo el comprador del pedido completado califica una vez y con un puntaje entero de 1 a 5; debe documentarse con captura del registro de calificación.

Estado de desarrollo: parcialmente implementado; aceptación no verificada con evidencia real suficiente.

El registro de la calificación en sí tiene una base de código razonable (validación de rol, pertenencia, estado y restricción de unicidad a nivel de base de datos), pero dos cosas impiden declarar RF-09 aceptado: el promedio del vendedor que el propio SRS exige mostrar nunca se calcula en el código, y casi ninguna rama del servicio calificarPedido() cuenta con una prueba real — solo se probó la validación del puntaje de forma aislada.

4. Requerimiento — RF-14: Editar perfil de usuario

Elemento de trazabilidad	Análisis
SRS — definición	CU-14 “Editar perfil de usuario” (RF-14): el usuario con sesión activa puede modificar nombre, correo, teléfono, fotografía y dirección opcionales; el vendedor también edita el nombre y la ubicación de su negocio. El rol no es editable y los pedidos históricos conservan sus datos.
Caso de uso	CU-14, disparado desde “Mi perfil”. El flujo principal tiene 7 pasos y 8 alternativas (correo duplicado, foto inválida, nombre vacío, negocio incompleto).
Búsqueda de evidencia backend	backend/src/app.js solo monta tres routers: /api/imagenes, /api/productos y /api/pedidos. No existe ningún router, controlador ni servicio para /api/perfiles o equivalente en todo el repositorio.
Búsqueda de evidencia frontend	No existe ninguna página “Mi perfil” entre las páginas de la aplicación (Auth, EditProduct, Home, Login, MyOrders, NotFound, ProductDetail, ProductList, PublishProduct, RecoverPassword, Register, UpdatePassword). AuthContext.jsx únicamente lee el perfil al iniciar sesión (supabase.from(“perfiles”).select(“*”)...single()) y permite cambiar la contraseña (actualizarContrasena); no existe ninguna función que escriba sobre la tabla perfiles desde el cliente.
Lo único relacionado que sí existe	supabase/migrations/001_init_schema.sql define la política de seguridad a nivel de fila “Perfiles: cada quien ve y edita el suyo” (using (auth.uid() = id)) sobre la tabla perfiles. Esta política permitiría la edición si el frontend la invocara, pero ningún código del cliente lo hace.
Pruebas reales ejecutadas	Ninguna. No se encontró código que implemente este requisito, por lo que no existe nada que una prueba automatizada pueda ejercitar.
Criterio de aceptación	Solo el titular modifica su perfil; el vendedor puede completar el nombre y la ubicación de su negocio desde ahí. No puede documentarse con evidencia porque la funcionalidad no existe en el código revisado.

Estado de desarrollo: no implementado.

A diferencia de los demás requisitos revisados en este proyecto, RF-14 no tiene ningún código de aplicación que lo respalde: ni ruta de backend, ni página de frontend, ni función de servicio. Lo único que existe es la política de base de datos que lo permitiría. Esto es una diferencia real entre la planificación del Sprint 6 (“Renzo: habilitar edición de perfil…”) y el estado actual del repositorio, y debería comunicarse al equipo antes de dar por aceptado este requisito.

5. Matriz de casos de prueba

Los siguientes casos derivan del criterio de cumplimiento de la planificación y de las ramas de los flujos de CU-06 y CU-14 en el SRS. A diferencia de los Sprints 4 y 5, la mayoría de los casos de este sprint quedan pendientes: en RF-09 por falta de pruebas sobre código que sí existe, y en RF-14 por falta de código que implemente el requisito.

ID	Requisito	Prueba y resultado	Resultado
CP-S06-01	RF-09	El puntaje de una calificación debe ser un entero entre 1 y 5; un valor fuera de rango es rechazado.	**PASS — evidencia real**
CP-S06-02	RF-09	Calificar un pedido propio en estado completado registra la calificación (puntaje y comentario opcional).	**Pendiente de ejecución**
CP-S06-03	RF-09	No se puede calificar un pedido que pertenece a otro comprador.	**Pendiente de ejecución**
CP-S06-04	RF-09	No se puede calificar un pedido que todavía no está en estado completado.	**Pendiente de ejecución**
CP-S06-05	RF-09	No se puede calificar dos veces el mismo pedido (restricción única a nivel de base de datos).	**Pendiente de ejecución**
CP-S06-06	RF-09	El promedio del vendedor se recalcula y se muestra tras registrar una calificación.	**Pendiente de ejecución**
CP-S06-07	RF-14	El usuario puede editar nombre, teléfono, foto y dirección desde “Mi perfil”.	**Pendiente de ejecución**
CP-S06-08	RF-14	Solo el titular puede modificar su propio perfil.	**Pendiente de ejecución**
CP-S06-09	RF-14	El vendedor puede editar el nombre y la ubicación de su negocio desde el perfil.	**Pendiente de ejecución**
CP-S06-10	RF-14	Un correo duplicado o un nombre vacío son rechazados al guardar el perfil.	**Pendiente de ejecución**

Resultados reales de la ejecución de pruebas (02/11/2026)

Suite	Archivos	Resultado
Backend — completo (con variables de entorno de prueba)	10 archivos	91 de 91 pruebas superadas
Backend — relacionadas con calificación (filtro “califc”)	1 archivo, 1 caso	1 de 1 prueba superada
Frontend — completo	4 archivos	49 de 49 pruebas superadas
Frontend — relacionadas con perfil o calificación	0 archivos	No se encontró ninguna prueba frontend sobre estos requisitos

6. Verificación del Definition of Done

Criterio	Estado	Evidencia / observación
Criterios de aceptación aceptados	No	RF-09 carece del cálculo del promedio del vendedor y de pruebas sobre el servicio de

Criterio	Estado	Evidencia / observación
		calificación; RF-14 no tiene código que lo implemente.
Pruebas y resultado registrados	Parcialmente	140 pruebas reales ejecutadas en total (91 backend + 49 frontend), pero solo 1 de ellas corresponde específicamente a RF-09 y ninguna a RF-14.
Cambios integrados	Parcialmente	El código de calificación (ruta, controlador, servicio, validación, restricción de base de datos) está en la rama principal; el de edición de perfil no existe.
Sin defectos críticos abiertos	No verificable	El repositorio no expone un registro de incidencias.
Evidencia guardada sin credenciales ni datos personales reales	Sí	Resultados de prueba y fragmentos de código citados en este informe; sin datos sensibles ni credenciales reales.

7. Hallazgos y acciones recomendadas

N.°	Hallazgo	Acción recomendada	Responsable
H-01	RF-14 (editar perfil de usuario) no tiene ningún código de aplicación: ni ruta de backend, ni página de frontend, ni servicio. Solo existe la política de base de datos que lo permitiría.	Implementar la ruta PATCH /api/perfiles (o equivalente) y la página “Mi perfil” en el frontend, siguiendo el flujo de CU-14 del SRS, antes de declarar el Sprint 6 cerrado.	Renzo
H-02	El paso 8 de CU-06 (“recalcula el promedio del vendedor”) no está implementado en ningún punto del código; el detalle del producto no muestra el promedio del vendedor pese a que el SRS y el propio diagrama de CU-04 lo contemplan.	Agregar una vista o consulta que calcule el promedio de puntaje por vendedor a partir de la tabla calificaciones, y exponerla en producto.service.js junto con los demás datos del vendedor.	Brant
H-03	El servicio calificarPedido() no tiene ninguna prueba real que lo ejercite (ni mock ni PGlite); solo se probó la validación aislada del puntaje.	Agregar pruebas a pedido.service.test.js (mock) y a database.test.js (PGlite) que cubran calificación válida, pedido ajeno, pedido no completado y calificación repetida.	Renzo / Brant
H-04	Sin un archivo .env configurado, 3 de 10 suites del backend no cargan (mismo hallazgo que en los informes de Sprint 3, 4 y 5).	Documentar variables de entorno de prueba (sin secretos reales) para CI, o versionar un .env.test de ejemplo.	Renzo
H-05	El Sprint Backlog planificado no coincide con el estado real del repositorio: todo el código corresponde a un único commit, y en este sprint parte de lo planificado (RF-14) directamente no se construyó.	Mantener commits incrementales por sprint en adelante para sustentar trazabilidad cronológica real y detectar brechas como esta antes de la entrega.	Ambos

8. Conclusiones

- RF-09 tiene una base de código razonable para registrar una calificación (validación de rol, pertenencia, estado completado y restricción de unicidad a nivel de base de datos), pero el promedio del vendedor que el propio SRS exige mostrar no está implementado en ningún lugar del código revisado.
- La cobertura de pruebas reales de RF-09 es mínima: de todo el servicio calificarPedido(), solo la validación aislada del puntaje (entero de 1 a 5) cuenta con una prueba real; los demás comportamientos (pedido ajeno, pedido no completado, calificación duplicada) no están probados, aunque sí están programados.
- RF-14 (editar perfil de usuario) no está implementado: no existe ruta de backend, página de frontend ni función de servicio para esta funcionalidad. Solo existe la política de base de datos que la permitiría si el cliente la invocara.
- De los diez casos de prueba identificados para este sprint, solo uno cuenta con evidencia real de ejecución (PASS); los nueve restantes quedan pendientes — seis por falta de pruebas sobre código existente (RF-09) y cuatro por falta de código que implementar (RF-14).
- Este sprint contrasta con los informes de Sprint 4 y 5, donde la mayor parte de lo planificado ya estaba construido y probado: aquí hay una brecha real entre el Sprint Backlog y el repositorio que el equipo debería resolver antes de la exposición o la entrega final.
- Las 140 pruebas automatizadas reales que siguen pasando (91 backend + 49 frontend) no cambian, porque provienen del mismo commit único ya evaluado en los informes anteriores; no se agregó ninguna prueba nueva relacionada con este sprint.

9. Bibliografía y fuentes técnicas

- Grupo G01. (2026). Planificación de Sprints de ÓrbiKa, versión 1.0, 12 de octubre de 2026: Sprint Backlog, sprints 5-6.
- Grupo G01. (2026). Informe de Trazabilidad — Sprint 5 ÓrbiKa (documento de referencia para el formato de este informe).
- Grupo G01. (2026). Especificación de requisitos de software (SRS), RF-09/CU-06 y RF-14/CU-14.
- Repositorio ORBIKA. (2026). supabase/migrations/001_init_schema.sql (tabla calificaciones, política “Perfiles: cada quien ve y edita el suyo”).
- Repositorio ORBIKA. (2026). backend/src/services/pedido.service.js, backend/src/validation/pedido.validation.js, backend/src/controllers/pedido.controller.js, backend/src/routes/pedido.routes.js, backend/src/app.js.
- Repositorio ORBIKA. (2026). backend/src/services/producto.service.js (ausencia de promedio de vendedor en la consulta de detalle).
- Repositorio ORBIKA. (2026). backend/test/validation/pedido.validation.test.js, backend/test/services/pedido.service.test.js, backend/test/sql/database.test.js.
- Repositorio ORBIKA. (2026). frontend/src/context/AuthContext.jsx y listado de páginas en frontend/src/pages/ (ausencia de una página de perfil).

Repositorio: https://github.com/renzoloyola/G01_CSW_ORBIKA